



Compliance, Quality Assurance & Tests

2025

Maven JUnit

2025

Conteúdo

1. Conceitos do Maven
2. JUnit
 1. Introdução
 2. Revisão – Testes unitários
 3. Anatomia de um teste JUnit
 4. Anotações
 5. Assertions
 6. Exceptions

Conceitos de Maven

Ferramenta utilizada para gerenciar projetos em Java e simplificar a vida do programador.

Auxilia no ciclo de desenvolvimento incluindo:

Compilação;

Controle de bibliotecas;

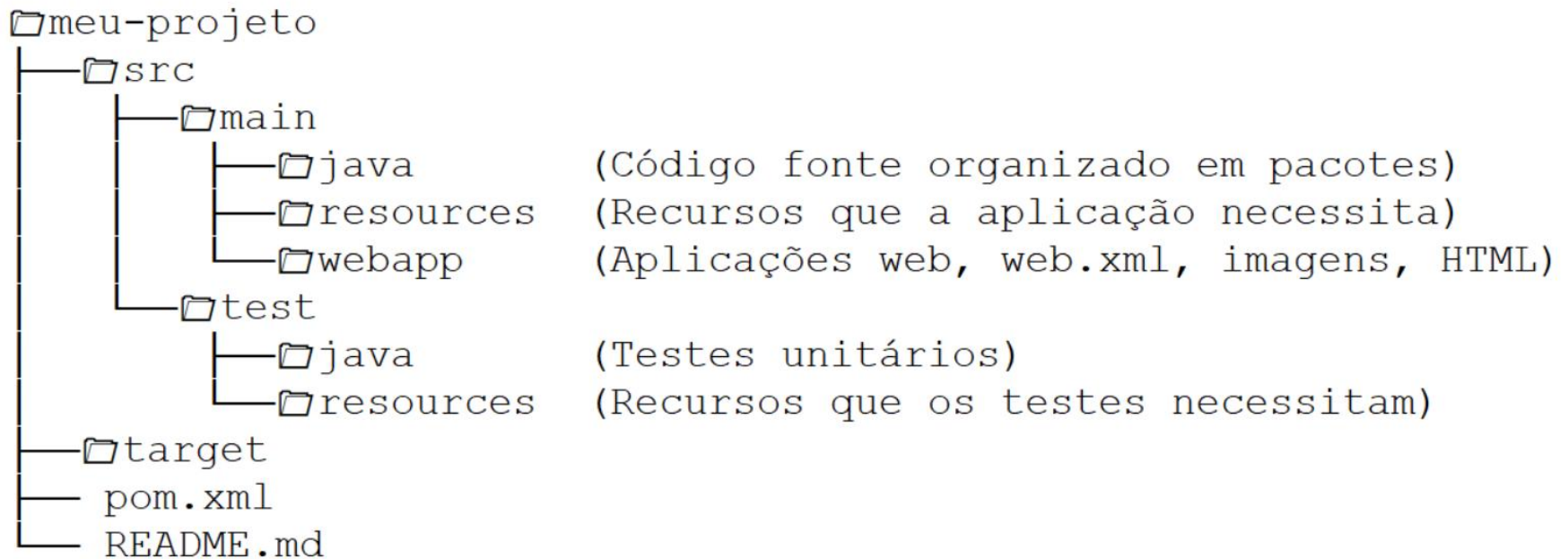
Distribuição;

Relatórios estatísticos.

O projeto nasceu a partir das dificuldades encontradas principalmente em gerenciar a compilação de projetos e no controle de bibliotecas.

Conceitos de Maven

Cada diretório de estrutura tem um propósito definido:



Conceitos de Maven

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4     http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6
7   <groupId>com.exemplo</groupId>
8   <artifactId>meu-projeto</artifactId>
9   <version>1.0.0</version>
10  <name>Meu Projeto</name>
11  <description>Um exemplo simples de projeto Maven</description>
12
13  <build>
14    <sourceDirectory>src/main/java</sourceDirectory>
15    <testSourceDirectory>src/test/java</testSourceDirectory>
16    <plugins>
17      <plugin>
18        <groupId>org.apache.maven.plugins</groupId>
19        <artifactId>maven-compiler-plugin</artifactId>
20        <version>3.8.1</version>
21        <configuration>
22          <source>1.8</source>
23          <target>1.8</target>
24        </configuration>
25      </plugin>
26      <plugin>
27        <groupId>org.apache.maven.plugins</groupId>
28        <artifactId>maven-jar-plugin</artifactId>
29        <version>3.2.0</version>
30        <configuration>
31          <archive>
32            <manifest>
33              <addClasspath>true</addClasspath>
34              <mainClass>com.exemplo.App</mainClass>
35            </manifest>
36          </archive>
37        </configuration>
38      </plugin>
39    </plugins>
40  </build>
41
42  <dependencies>
43    <dependency>
44      <groupId>junit</groupId>
45      <artifactId>junit</artifactId>
46      <version>4.13.1</version>
47      <scope>test</scope>
48    </dependency>
49  </dependencies>
50</project>
```

A configuração do Maven se baseia em um único arquivo, chamado **pom.xml**, que contém os metadados de um projeto.

É a partir deste arquivo que toda a magia por trás do Maven acontece.

JUnit

O JUnit é um dos mais populares frameworks de teste open source para Java, que permite a criação de testes unitários automatizados.

O JUnit 5 é a versão mais recente da ferramenta e trouxe muitas mudanças em relação às versões anteriores:

- Suporte para Java 8 e versões posteriores, com expressões lambda;
- Suporte para testes parametrizados;
- Nova arquitetura que permite extensões personalizadas e suporte a diferentes motores de execução de testes;
- Melhorias de desempenho;
- Suporte para anotações de teste personalizadas;
- Suporte a testes assíncronos.

Revisão – Testes Unitários

Testes unitários são uma prática de engenharia de software que consiste em escrever código automatizado para testar pequenas unidades de código, geralmente funções ou métodos individuais, para garantir que eles funcionem como esperado.

Testes unitários são geralmente escritos por desenvolvedores e executados durante o processo de desenvolvimento de software.

- Ajudam a detectar bugs o mais cedo possível no processo de desenvolvimento.
- Reduzem o tempo de depuração, pois tornam mais fácil identificar e corrigir problemas de código.
- Ajudam a documentar o código e as regras de negócio, pois descrevem explicitamente o comportamento esperado de cada unidade de código → **TDD**.
- Facilitam a manutenção do software, pois permitem alterações “sem medo”.

JUnit – Anatomia de um teste

Anotações (Annotations)

São usadas para definir e configurar os métodos de teste. Algumas das mais importantes são:

- @Test
- @BeforeEach / @BeforeAll
- @AfterEach / @AfterAll
- @ParameterizedTest
- @Disabled
- @Timeout

Classe de Teste

Sugere-se sempre usar o mesmo nome da Classe que está sendo testada adicionando o sufixo "Test"

Método de Teste

Sugere-se usar um nome bastante descritivo sobre o cenário ou comportamento esperado a ser testado.

Lógica do teste

Implementações do código usado para execução do passo a passo do caso de teste.

```
public class CalculatorTest {  
    @Test  
    public void testAddition() {  
        Calculator calculator = new Calculator();  
        int result = calculator.add(2, 3);  
  
        assertEquals(5, result);  
    }  
}
```

Asserções (Assertions)

Verificam se o resultado da lógica implementada corresponde ao esperado. Algumas das mais importantes são:

- assertEquals() / assertNotEquals()
- assertTrue() / assertFalse()
- assertThrows() / assertDoesNotThrow()
- assertNull() / assertNotNull()

Até aqui alguém tem
alguma pergunta?



Operações de uma calculadora

```
1 package Calculadora;
2
3 public class Calculadora {
4
5     public int somar(int a, int b) {
6         return a + b;
7     }
8
9     public int subtrair(int a, int b) {
10        return a - b;
11    }
12
13    public int multiplicar(int a, int b ) {
14        return a * b;
15    }
16
17    public int dividir(int a, int b) {
18        return a / b;
19    }
20
21 }
```

Atenção ⚠

CÓDIGO REPETITIVO

```
10 public class CalculadoraTest {
11
12•  @Test
13    void testeSomar() {
14
15        Calculadora calc = new Calculadora();
16
17        int a = 3;
18        int b = 2;
19        int resultado = calc.somar(a, b);
20
21        assertEquals(a+b, resultado);
22    }
23
24•  @Test
25    void testeSubtrair() {
26
27        Calculadora calc = new Calculadora();
28
29        int a = 3;
30        int b = 2;
31        int resultado = calc.subtrair(a, b);
32
33        assertEquals(a-b, resultado);
34    }
35
36•  @Test
37    void testeMultiplicar() {
38
39        Calculadora calc = new Calculadora();
40
41        int a = 3;
42        int b = 2;
43        int resultado = calc.multiplicar(a, b);
44
45        assertEquals(a*b, resultado);
46    }
47
48•  @Test
49    void testeDividir() {
50
51        Calculadora calc = new Calculadora();
52
53        int a = 3;
54        int b = 2;
55        int resultado = calc.dividir(a, b);
56
57        assertEquals(a/b, resultado);
58    }
59 }
```

Anotações

@BeforeEach	Indica que o método deve ser executado <u>antes de cada método</u> de teste dentro da classe atual.
@BeforeAll	Indica que o método deve ser executado <u>uma única vez antes de todos</u> os métodos de teste dentro da classe atual.
@AfterEach	Indica que o método deve ser executado <u>após cada método</u> de teste dentro da classe atual.
@AfterAll	Indica que o método deve ser executado <u>antes uma única vez após</u> todos os métodos de teste dentro da classe atual.

Exercício - Refatorando

```
11 public class CalculadoraTest {
12
13     private Calculadora calc;
14
15     @BeforeEach
16     public void inicializar() {
17         this.calc = new Calculadora();
18     }
19
20     @Test
21     public void testeSomar() {
22
23         //calc = new Calculadora();
24
25         int a = 3;
26         int b = 2;
27         int resultado = calc.somar(a, b);
28         |
29         assertEquals(a+b, resultado);
30     }
31 }
```

Até aqui alguém tem
alguma pergunta?



Assertions

As Asserções (*Assertions*) são métodos utilitários do JUnit que verificam se uma dada condição ou comportamento do código está de acordo com o que era esperado.

Esses métodos são acessados pela classe `org.junit.jupiter.api.Assertions` no JUnit 5.

- Ordem dos parâmetros:
 <esperado>, <atual>

- **Import Static**

Por questões de legibilidade, recomenda-se o uso de importação estática da respectiva classe, para que o método seja referenciado diretamente sem o prefixo de sua classe representativa.

Assertions - Exemplos

<code>assertArrayEquals</code>	Verifica se as matrizes passadas nos parâmetros <i>expected</i> e <i>actual</i> são iguais.
<code>assertEquals</code> <code>assertNotEquals</code>	Verifica se os objetos passados nos parâmetros <i>expected</i> e <i>actual</i> são iguais ou diferentes.
<code>assertTrue</code> <code>assertFalse</code>	Verifica se dada condição retorna o booleanos Verdadeiro <i>True</i>) ou Falso (<i>False</i>).
<code>assertNull</code> <code>assertNotNull</code>	Verifica se um dado objeto é ou não nulo (<i>null</i>).
<code>assertThrows</code> <code>assertDoesNotThrow</code>	Permite verificar se um executável (<i>executable</i>) lança uma exceção do tipo especificado, ou não lança nenhuma exceção.
<code>assertAll</code>	Permite a criação de asserções agrupadas, onde todas são executadas e suas falhas reportadas em conjunto.
<code>fail</code>	Ao ser executado, atribui falha ao teste imediatamente, adicionando mensagem opcional de falha.

Até aqui alguém tem
alguma pergunta?



Referências:

<https://www.devmedia.com.br/gerenciando-projetos-com-maven>

<https://www.devmedia.com.br/assertions-em-java>

<https://pt.wikipedia.org/wiki/JUnit>

<https://www.baeldung.com/junit-assertions>

<https://junit.org/junit5/docs/current/user-guide/>

<https://www.baeldung.com/junit-4-rules>

FIAP